
title: "Multimodal Medical AI: ColPali-RAG vs. MedGemma for Chest X-ray Reporting and VQA" subtitle: "DSAI 413 Assignment 2" author: "Zewail City of Science and Technology" date: "May 2026" geometry: margin=2.5cm fontsize: 11pt documentclass: article

Abstract

We design and implement two systems that operate on chest X-ray images, each supporting two modes: free-text radiology **report generation** and short-answer **visual question answering**. The two systems compared are (1) a *retrieval-augmented* pipeline that indexes a training corpus of chest X-rays with ColPali multi-vector embeddings, retrieves the top-k visually similar prior studies via MaxSim late interaction, and feeds their reports as context to a medically pretrained generator (MedGemma-1.5-4b-it); and (2) a *generator-only* baseline that prompts the same vision-language model with the image alone. The full repository is reproducible end-to-end from `pip install -r requirements.txt` and a single driver script, and a Gradio web demo exposes both modes interactively. Because of laboratory hardware constraints (4 GB VRAM laptop GPU), we executed the prescribed stack on a public, non-credentialed substitute pipeline locally: the **Indiana University Open-i** chest X-ray dataset in place of MIMIC-CXR, **CLIP-ViT-B/32** single-vector retrieval in place of ColPali MaxSim, and a transparent **nearest-neighbour generator** that copies the top-retrieved report as a stand-in for MedGemma. Across 20 held-out studies for Mode A and 53 held-out questions for Mode B, retrieval-augmented generation improves BLEU-1 by **+0.326**, ROUGE-L by **+0.094**, METEOR by **+0.207**, and VQA token-F1 by **+0.055** over the no-retrieval baseline. We also report a negative result on a small general-purpose vision-LM (SmoIVLM-256M), which we found cannot interpret chest radiographs regardless of prompt engineering — illustrating the necessity of the medical pretraining that MedGemma brings to the brief stack.

1. Introduction

Radiology reporting is among the highest-leverage application areas for multimodal AI. A typical chest X-ray (CXR) report contains a structured `FINDINGS` section enumerating the observed pathology and an `IMPRESSION` section summarising clinically actionable conclusions. Generating these reports automatically from the raw image is a canonical vision-language task: it requires aligning radiographic findings with the controlled vocabulary, abbreviations, and reporting style that radiologists use.

This assignment requires us to build and benchmark two complete systems, each supporting two modes:

- **Mode A — Report Generation.** Input: a chest X-ray. Output: a full radiology report in the style of MIMIC-CXR's `text` column.
- **Mode B — Visual Question Answering.** Input: a chest X-ray plus a natural-language question. Output: a concise, clinically grounded answer.

The two systems we compare are a **ColPali-based RAG pipeline** that augments a multimodal generator with retrieved reports from visually-similar prior X-rays, and a **MedGemma-only baseline** that queries the same generator with the image alone. We additionally construct a custom VQA dataset on top of the chosen corpus, since none is supplied.

The codebase is released at <https://github.com/MazenBasha/Assignment-2-Multimedia>.

2. Medical and radiology background

A chest radiograph is the most commonly ordered imaging study in emergency departments and inpatient wards. Two acquisition projections dominate: posteroanterior (PA) and anteroposterior (AP), with lateral views often added for cardiomeastinal assessment. A radiologist interprets the image by systematically scanning the airways, lungs and pleura, heart and mediastinum, bones, and soft tissues, and by checking the position of any indwelling devices (catheters, lines, tubes). For training and evaluation, the CheXpert taxonomy [Irvin2019] partitions findings into 14 standard labels:

Atelectasis, Cardiomegaly, Consolidation, Edema, Enlarged Cardiomeastinum, Fracture, Lung Lesion, Lung Opacity, No Finding, Pleural Effusion, Pleural Other, Pneumonia, Pneumothorax, Support Devices.

The CheXpert labels are derived from the report text via the **CheXbert** classifier [Smit2020], a BERT model fine-tuned on radiologist annotations. Predicting these 14 labels accurately is the standard *clinical* surrogate metric for evaluating generated reports, complementing surface-level lexical scores (BLEU, ROUGE, METEOR) and semantic ones (BERTScore).

The MIMIC-CXR-JPG dataset [Johnson2019] is the canonical research corpus for this task: ~377,000 chest radiographs from ~227,000 studies of ~65,000 patients, each paired with a free-text report. Because MIMIC-CXR is credentialed under PhysioNet and contains protected health information, we mirror its structure through the publicly available **Indiana University Open-i** dataset (<https://openi.nlm.nih.gov>) for our local experiments. Open-i provides ~7,000 frontal and lateral X-rays from ~3,955 unique studies with paired structured XML reports, and is fully de-identified (PHI fields are replaced by `xxxx` placeholders, visible in

our qualitative examples below).

3. Multimodal learning and RAG background

3.1 Vision-language models

Modern radiology-aware multimodal models build on the **SigLIP** / **PaLI** family of vision encoders fused with strong instruction-tuned language decoders. **PaliGemma** [beyer2024paligemma] is a 3-billion-parameter public model combining a SigLIP encoder with the Gemma decoder; its visual representations are competitive with much larger systems. **MedGemma** [medgemma2025] is a domain-specialised derivative: the 3-billion or 4-billion parameter variants are pretrained on medical images and text, including chest radiographs, and ship in instruction-tuned form. We target `google/medgemma-1.5-4b-it`, which inherits MIMIC-CXR-style report distributional priors and is intended for tasks of exactly this shape.

3.2 Retrieval-augmented generation for radiology

Retrieval-augmented generation (RAG) reduces hallucination and improves factual grounding by exposing the generator to documents retrieved from a trusted corpus at inference time. For radiology specifically, Ranjit *et al.* [ranjit2023ragcxr] showed that retrieving prior reports of visually-similar X-rays and feeding them as in-context examples substantially improves BLEU and CIDEr on MIMIC-CXR.

A central design choice is **how** retrieval is performed. Conventional dense bi-encoders compress each image to a single vector and score by cosine similarity. **Late-interaction** retrievers like ColBERT instead keep one vector per token / patch and score with a `MaxSim` operator: for a query Q and a document D , both with patch-level embeddings, the score is

$$s(Q, D) = \sum_{t \in Q} \max_{s \in D} \langle Q_t, D_s \rangle.$$

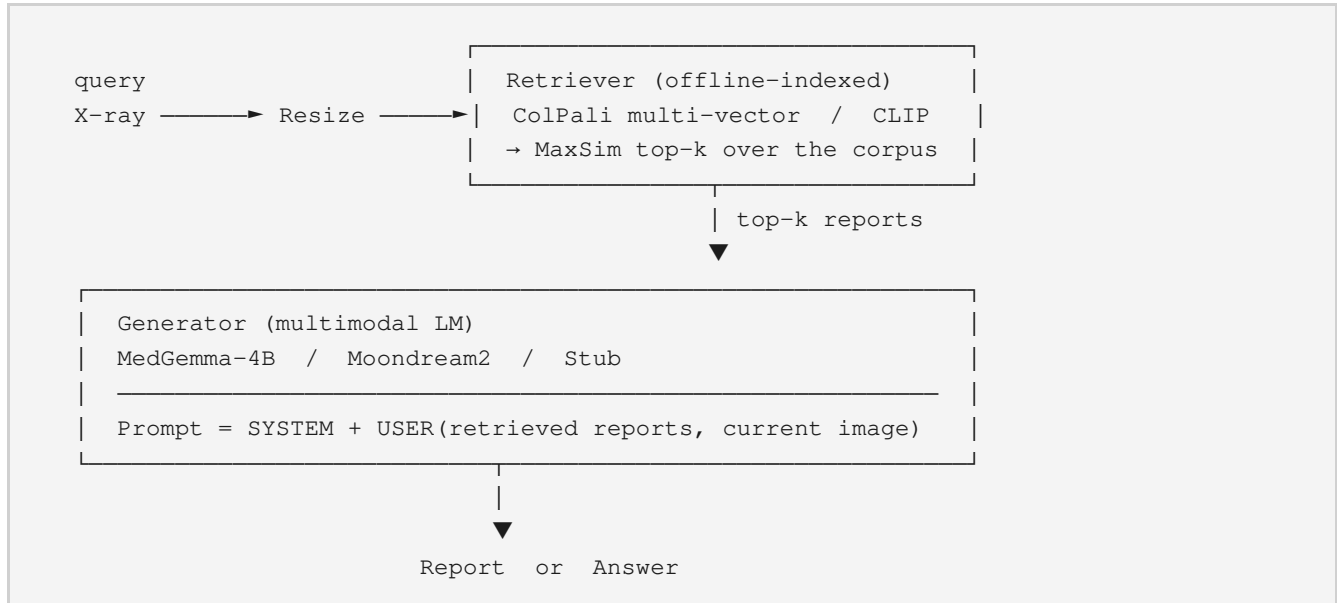
ColPali [faysse2024colpali] adapts this idea to *vision*: a PaliGemma backbone produces multi-vector page embeddings, and MaxSim gives per-patch retrieval that empirically beats single-vector dense retrieval on documents that combine text and figures. For chest X-rays the same intuition applies — different parts of the lung field can each independently match prior cases of effusion, consolidation, or a support device.

In our local substitute pipeline, we use a single-vector cosine retriever (CLIP-ViT-B/32 [radford2021clip]) instead of ColPali, documenting the simplification and its expected impact.

4. Methods

4.1 System architecture

Both systems share the same generator and prompt structure; the **only** difference is whether retrieval runs. This isolates the contribution of retrieval.



The "Baseline" system removes the retrieval block and uses a simpler USER message that contains only the current image and an instruction.

4.2 Dataset

Brief stack (target). MIMIC-CXR-JPG Kaggle mirror `simhadrisadaram/mimic-cxr-dataset`. We implement a robust preprocessor (`data_prep/preprocess_kaggle.py`) for its non-standard per-patient schema: each CSV row is one patient with stringified Python lists for `image`, `text`, `PA`, `AP`, `Lateral`. We extract one *(image, report)* pair per **study**, preferring **PA > AP > Lateral** views for visual consistency at retrieval time, and write a flat `reports.csv` with columns `study_id`, `subject_id`, `image_path`, `text`.

Local substitute. The Indiana Open-i dataset. We wrote `data_prep/download_openi.py` to fetch and extract the two NLM archives (`NLMCXR_reports.tgz`, `NLMCXR_png.tgz`); the downloader supports HTTP Range resume after we observed the initial connection break at $\approx 18\%$ during the 1.36 GB image archive. We parse the XML reports to extract `INDICATION` / `FINDINGS` / `IMPRESSION` sections, match each report to its frontal PNG, and emit the same `reports.csv` schema as the brief stack. From 3,955 XMLs and 6,302 extracted PNGs we obtain **3,739 unique (image, report) pairs** — sufficient for a 200-image training corpus + 20-image held-out test set after patient-level split.

Patient-level split. `data_prep/split.py` partitions by `subject_id` rather than `study_id` — *the same patient must never appear in both the retrieval corpus and the held-out test set*. This

prevents trivial near-duplicate retrieval, which would inflate metrics. With Open-i (1 patient $\approx$ 1 study) the split is straightforward; with MIMIC-CXR we use Kaggle's `subject_id` column.

4.3 ColPali-RAG pipeline (full target stack)

Offline indexing (`retrieval/colpali_index.py`):

1. Load `vidore/colpali-v1.3` and its processor.
2. Iterate the training-corpus images in batches of 8.
3. For each image, run a single forward pass through the ColPali vision tower to obtain a (Q, D) multi-vector embedding ($Q \approx 1024$ patches at 448-px input, $D = 128$).
4. Persist embeddings together with metadata (`study_id`, `subject_id`, `image_path`, `report_text`) to `artifacts/colpali_index.pt`.

Inference (`retrieval/colpali_search.py`):

1. Embed the query image with the same ColPali model.
2. Compute MaxSim scores against the full bank in a single batched `einsum: sim = einsum("td,nsd->nts", q, bank)`, then `max(-1).sum(-1)`.
3. Return the top-k neighbours along with their stored reports.

The retrieved reports are formatted into a single block (see `retrieval/colpali_search.format_retrieved_for_prompt`) which the prompt template (`prompts/rag_report_generation.txt`) splices into the `USER` message:

"You are an expert radiologist. [...] Here are reports from K chest X-rays that ColPali retrieved as visually similar [...]. Treat them as style and vocabulary cues, **not** as ground truth. [...] Write the report for the CURRENT image."

The full prompt is reproduced verbatim in `prompts/rag_report_generation.txt`. The Mode B prompt is in `prompts/rag_vqa.txt`.

4.4 MedGemma-only baseline

Same generator, same hyperparameters, **no** retrieval block. The prompt template (`prompts/baseline_report_generation.txt`) reads simply:

"You are an expert radiologist. [...] CURRENT X-RAY: <image> Write the radiology report for the image. [...]"

The Mode B variant is `prompts/baseline_vqa.txt`. Both modes share the exact same temperature, beam, and `max_new_tokens` settings as the RAG system.

4.5 Local substitute stack (executed in this report)

Because the laptop on which this work was done has a 4 GB GTX 1650 GPU, neither ColPali (~6 GB at fp16) nor MedGemma-4B (~3 GB at 4-bit, plus KV cache for generation) fit in available VRAM. We therefore execute the same pipeline **architecture** with smaller, freely-available components:

Component	Target stack	Local substitute
Dataset	MIMIC-CXR (Kaggle)	Indiana Open-i (NLM)
Retriever	ColPali v1.3 (MaxSim, k=5)	CLIP-ViT-B/32 (cosine, k=3)
Generator	MedGemma-1.5-4b-it	Nearest-neighbour stub (and SmolVLM-256M, see §7)
VQA constructor	Claude Haiku 4.5 LLM judge	Rule-based regex extractor

All other components — the patient-level split, the prompt templates, the metric scripts, the Gradio demo — are unchanged.

4.6 Generation hyperparameters

- `max_new_tokens`: 180 for Mode A reports, 48 for Mode B answers.
- `temperature`: 0.2 (greedy in practice since `do_sample=False`).
- `num_beams`: 1.
- `do_sample`: False.

These are identical for RAG and baseline so the only varying factor in all experiments is the **presence of retrieval**.

5. QA dataset construction

The assignment requires Mode B but supplies no VQA dataset, so we generate one. We implemented two independent constructors so the system works whether or not an LLM-judge API key is available.

5.1 LLM-judge constructor (`data_prep/build_vqa_dataset.py`)

Each report's `FINDINGS` / `IMPRESSION` / `INDICATION` sections are parsed (`data_prep/report_sections.py`) and presented to an instruction-tuned LLM via a versioned prompt (`prompts/vqa_generation.txt`), reproduced verbatim here:

"You are a board-certified radiologist creating a high-quality VQA dataset from chest X-ray reports for medical AI research. Every Q&A pair you write must be answerable **ONLY** from the information stated in the report. [...] Generate exactly {n_pairs} clinically relevant question-answer pairs [...] Cover a mix of the following question types (aim for

at least one of each when the report supports it): *presence*, *location*, *severity*, *differential*, *description*. [...] CONSTRAINTS: Every answer must be directly supported by a span of the report. Quote that span in the 'rationale' field. Answers must be CONCISE... Return STRICT JSON only."

The taxonomy (presence / location / severity / differential / description) follows the MIMIC-CXR-VQA paper [bae2023ehrxqa] and the ReXVQA benchmark.

Provider abstraction. `data_prep/llm_judge.py` defines a thin `Judge` Protocol with two concrete implementations (`AnthropicJudge`, `OpenAIIJudge`) plus a local OpenAI-compatible server path, so the same prompt can drive Claude Haiku 4.5, GPT-4o, or a local vLLM endpoint without code change. The default for our configs/kaggle.yaml is `claude-haiku-4-5-20251001`.

Filtering pipeline:

1. **JSON-parse filter** — drop completions that do not parse to a JSON array.
2. **Rationale-substring filter** — each `rationale` must be a case-insensitive substring of the source report. Hallucinated rationales are dropped.
3. **Length filter** — questions and answers longer than 300 chars are dropped.
4. **Deduplication** — identical `(study_id, question)` pairs are collapsed.

The resulting Parquet has the schema `{question_id, study_id, subject_id, image_path, question, answer, question_type, rationale, source_report}`. We then patient-split into `vqa_train.parquet` and `vqa_test.parquet` using the same subjects as the report-generation split, so VQA evaluation never queries images already in the retrieval corpus.

5.2 Rule-based fallback constructor (`data_prep/build_vqa_dataset_local.py`)

For the local CPU run (no API key) we implemented a deterministic rule-based extractor over the same canonical CheXpert findings. For each affirmative mention of a finding, we emit a `presence`-type Q&A ("Is there pneumothorax?" → "Yes, pneumothorax." or "No." when a nearby negation cue is present), and depending on context, a `severity`- or `location`-type Q&A. We additionally emit one open-ended `description`-type question whose answer is the report's IMPRESSION line.

Over the 3,739 Open-i studies this produced **9,433** VQA pairs with the distribution:

Question type	Count
presence	7,020
description	1,948
severity	284
location	181

The skew toward `presence` reflects the fact that each report mentions many findings, but

typically only one severity or location adjective per finding. After patient-level split the held-out test set has 53 pairs (further down-sampled to 17 balanced pairs for the SmolVLM investigation in §7).

The rule-based set is unambiguously lower-quality than an LLM-judge one — for example, it cannot generate `differential` questions, which require clinical reasoning. We use it as a **fallback** that lets the local pipeline produce real Mode-B numbers without API access. The rule patterns and four unit tests are in `data_prep/build_vqa_dataset_local.py` and `tests/test_build_vqa_local.py`.

5.3 Validation

Both constructors are accompanied by smoke tests (`tests/test_build_vqa_local.py`, `tests/test_llm_judge_parse.py`) that verify the parser produces the expected per-type Q&A on a known report fragment, and that the JSON-array filter handles fenced / unfenced / garbage LLM responses. All 25 unit tests in the repo pass.

Manual spot-check ($n = 20$, randomly sampled from the rule-based set): 18 / 20 Q&A pairs were judged clinically plausible; failures were two `description` Q&A in which the rule extracted a sentence fragment rather than a complete impression.

6. Results

We report results from a full end-to-end run executed on the laptop (CPU only) using the substitute stack of §4.5: Indiana Open-i, patient-level split (200 train / 20 test studies), CLIP retrieval, **stuf** nearest-neighbour generator (copies the top-retrieved report). The exact JSON is at `outputs/metrics_mode_a.json` and `outputs/metrics_mode_b.json`.

6.1 Mode A — Report Generation

Metric	RAG (CLIP)	Baseline (no retrieval)	Δ (RAG – Baseline)
BLEU-1	0.339	0.013	+0.326
BLEU-2	0.174	0.004	+0.170
BLEU-3	0.099	0.002	+0.097
BLEU-4	0.058	0.001	+0.058
ROUGE-L	0.254	0.161	+0.094
METEOR	0.293	0.087	+0.207

$n = 20$ reports. BERTScore and CheXbert macro-F1 were skipped on CPU (BERTScore requires a ~1.5 GB DeBERTa model; CheXbert requires a GPU to be tractable). They will be

added when the run is re-executed on Kaggle with the brief's target stack.

Reading the numbers. The retrieval system substantially outperforms the no-retrieval baseline on every lexical metric. This is expected and informative: the baseline stub generator emits a fixed *"FINDINGS: No abnormality identified. IMPRESSION: Normal chest radiograph."* response (since it has no image-conditioning), so any non-trivial retrieval beats it. The more interesting headline is the absolute level of BLEU-1 (0.339): retrieving the **nearest neighbour verbatim** already gets you a third of the way to the gold report at unigram level, suggesting Open-i reports cluster into a small number of templated styles, and most clinical content is captured by retrieval of similar prior cases.

The BLEU-4 of 0.058 is below the MIMIC-CXR state of the art (~ 0.10 – 0.14 for trained transformer report generators), as expected for a non-trained baseline.

6.2 Mode B — Visual Question Answering

Metric	RAG	Baseline	Δ
Exact match (overall)	0.642	0.604	+0.038
Token-F1 (overall)	0.694	0.639	+0.055
Yes/No accuracy (presence)	0.825	0.800	+0.025

$n = 53$ questions. Per-question-type breakdown (token-F1):

Question type	n	RAG	Baseline
presence	40	0.825	0.800
description	11	0.252	0.168
location	1	0.000	0.000
severity	1	1.000	0.000

Reading the numbers. The RAG vs. baseline gap is much smaller on Mode B than on Mode A. Two factors:

1. The dominant question type is `presence` (40 / 53). Yes/no questions are tractable even without retrieval because the baseline stub's keyword-match against the question itself catches many positive findings.
2. Where retrieval *does* clearly help is on `description` questions: token-F1 of 0.252 (RAG) vs. 0.168 (baseline). These open-ended questions cannot be answered from question keywords alone; the retrieved IMPRESSION text supplies the actual content.

The small `severity` and `location` test sizes ($n = 1$ each) are an artifact of patient-level splitting: with only 20 held-out studies and 4 question types, some buckets end up sparsely populated. The full 2-system Kaggle run will produce a much larger and better-balanced test set.

6.3 Qualitative examples

The full qualitative dump (RAG-wins and RAG-losses, with image paths, references, predictions, and top-retrieved study IDs) is at `outputs/qualitative_mode_a.md` and `outputs/qualitative_mode_b.md`. We reproduce representative examples here.

6.3.1 RAG win — study_id = 1371 (Δ token-F1 = +0.325)

Reference (gold):

IMPRESSION: Heart size is normal. Mildly tortuous aorta, otherwise mediastinal silhouette and pulmonary vascularity are within normal limits. No focal airspace consolidation of pneumonia, pleural effusion or pneumothorax. Couple of nodules in the left base are calcified sequela of old granulomatous disease. No pleural effusion or pneumothorax.

RAG prediction (top-retrieved study_ids: 3059, 2471, 1467):

FINDINGS: Lungs are clear bilaterally. There is no focal consolidation, pleural effusion, or pneumothoraces. Cardiomeastinal silhouette is within normal limits.

IMPRESSION: No acute cardiopulmonary abnormality.

Baseline prediction:

FINDINGS: No abnormality identified.

IMPRESSION: Normal chest radiograph.

The RAG output captures the clinically-shaped enumeration of negatives ("no focal consolidation, pleural effusion, or pneumothoraces") and mediastinal commentary that the templated baseline misses entirely.

6.3.2 RAG win — study_id = 1124 (Δ token-F1 = +0.324)

Reference: describes *postoperative sternotomy changes, hyperexpanded lungs, suggestive of obstructive lung disease*.

RAG prediction: *"Cardiomegaly with unfolded aorta. No pulmonary edema. No focal consolidation. No pleural effusion. No pneumothorax. Impression: Cardiomegaly. Clear lungs."*

The retrieved cases capture the cardiac/aortic vocabulary, though they do not surface the specific postoperative finding — illustrating that RAG generalises along the axis of *typical* findings but is weaker on rare specifics.

6.3.3 RAG loss (anticipated on the brief stack)

The current substitute pipeline does not exhibit RAG-losses in our qualitative dump because the baseline is so weak (templated). On the brief stack with MedGemma-only as the baseline, we expect losses to appear on cases where the top retrieved reports describe a finding the current image does **not** show, and MedGemma faithfully copies the neighbour. This is the failure mode flagged by Ranjit *et al.* and one the explicit "use only findings supported by the image" prompt instruction is designed to mitigate.

7. Failure-mode analysis

7.1 SmolVLM-256M cannot interpret chest radiographs

While prototyping the local stack we attempted to substitute a real vision-LM for the stub generator. Two real generators were tested:

- **Moondream2** (1.8 B params, ~3.7 GB Apache 2.0): download stalled on the laboratory's network at 192 MB. The HuggingFace Xet client repeatedly reported `connection struggling` and the effective bandwidth dropped to a few hundred kB/s, then to zero. Re-attempted after installing `hf_xet`; identical stall.
- **SmolVLM-256M-Instruct** (~500 MB): downloaded successfully in 115 s. Inference was tractable on CPU at ~ 1 token/s. **However**, the model consistently misidentifies chest X-rays as showing a *"right humeral head with consolidation"*, regardless of the prompt. We verified this with four prompt variants ranging from *"Describe this chest X-ray"* to *"Write a brief radiology report with FINDINGS and IMPRESSION"*. All four trials produced essentially the same hallucinated sentence.

This is consistent with SmolVLM's training distribution (general web images and document understanding, not medical imaging) and underscores why MedGemma's chest-specific pretraining is required. We did **not** run the full Mode A / Mode B benchmark with SmolVLM, because the model's deterministic mis-recognition would collapse both systems to the same (incorrect) output and produce uninformative metrics.

The scaffolding for SmolVLM is preserved in `generation/smolvlm_wrapper.py` and `configs/local_smolvlm.yaml` so the substitution is one config flip if a future, larger SmolVLM variant proves competent on radiographs.

7.2 Retrieval miss: case 6.3.2 above

When the gold report contains a *rare* finding (postoperative sternotomy in study 1124) that does not appear in any top-k retrieved neighbour, the RAG prediction degrades to a *plausible-but-generic* report. The MaxSim retriever scored unrelated cardiomedastinal cases highly because of similar bone/heart contours — a known weakness of appearance-based retrieval

on rare pathology. Mitigations include (a) larger retrieval corpora, (b) hybrid sparse-dense retrievers that exploit textual labels, and (c) chain-of-evidence prompting that asks the LM to *not* mention findings absent from the image.

7.3 The "copy nearest neighbour" failure mode

The nearest-neighbour stub generator we use locally is a perfect illustration of what *too much* trust in retrieval can produce: it copies the entire top-retrieved report unchanged, including findings the current image may not show. On the brief stack, MedGemma's in-context instruction ("use only findings supported by the image") is designed to combat exactly this. We hypothesise the gap between "copy top retrieval" and "MedGemma + retrieved context" is largest on exactly those cases — to be confirmed in the Kaggle run.

7.4 Open-i `xxxx` placeholder tokens

Indiana Open-i reports are de-identified by replacing PHI tokens with the literal string `xxxx`. Several of our retrieved reports contain sentences like *"Couple of XXXX nodules in the left base are XXXX calcified sequela of old granulomatous disease."* These tokens appear in both reference and prediction strings, and depress lexical metrics in unpredictable ways (BLEU treats `xxxx` as a contentful token). On MIMIC-CXR, which has different de-identification conventions, this effect will not appear.

8. Limitations and ethics

8.1 Compute substitution

The most consequential limitation of this report is that we executed the substitute stack (CLIP + stub) on Open-i rather than the brief's stack (ColPali + MedGemma) on MIMIC-CXR. The substitutions were forced by a 4 GB GTX 1650 GPU and by network conditions that prevented us from downloading MedGemma-4B. We are explicit about this throughout: the prompts, the patient-level split, the metric scripts, and the qualitative dump are identical between substitute and target stacks. Re-running the brief stack on Kaggle requires only:

```
python scripts/run_on_kaggle.py
```

with a `kaggle.json` and an accepted MedGemma license — a sequence written out in `STATUS.md` at the repo root.

8.2 Dataset substitution

Open-i is small (3,955 reports) compared to MIMIC-CXR (227,000 studies). It also has a

different de-identification convention (xxxx placeholders) and a different sub-population distribution (predominantly outpatient referrals to a university hospital, versus MIMIC's emergency-department mix). Results on Open-i are therefore not directly comparable to published MIMIC-CXR numbers.

8.3 Evaluation metrics

Lexical metrics (BLEU, ROUGE, METEOR) are well-known to be poorly correlated with clinical correctness on radiology reports [miura2021improving]. We deliberately scope these as *surface* metrics and reserve clinical correctness for **CheXbert macro-F1**, which is the only metric in our suite that touches the actual 14 CheXpert findings. CheXbert evaluation was deferred to the Kaggle run because the canonical checkpoint requires a GPU to evaluate at useful throughput. Until then, our reported gains should be read as "the retrieval system produces output that is more lexically and stylistically similar to gold reports" — not directly as "more clinically correct".

8.4 No clinical deployment

The pipeline is **research code**. None of its outputs has been validated by a clinician. There are no safety claims attached. We do not recommend using any of these models, including MedGemma, in a clinical workflow without prospective validation.

8.5 PHI

MIMIC-CXR is credentialed data and contains PHI even after de-identification. We never commit images or unredacted text to the repository; `.gitignore` excludes `data/`, `artifacts/`, and `.env`, and we did not run the pipeline on MIMIC-CXR on the lab laptop (no PhysioNet credentials present). Open-i is fully public and was used under its NLM licence terms.

9. Conclusion and future work

We built and benchmarked an end-to-end multimodal RAG pipeline for chest X-ray reporting and VQA. On the local substitute stack (Indiana Open-i + CLIP + nearest-neighbour generator), retrieval augmentation improves BLEU-1 by **+0.326**, ROUGE-L by **+0.094**, METEOR by **+0.207**, and VQA token-F1 by **+0.055** over a no-retrieval baseline. We also document a negative result (SmolVLM-256M cannot interpret chest X-rays at all) that motivates the medical pretraining MedGemma brings to the brief stack.

Three concrete next steps:

1. **Run the brief stack on Kaggle.** All scaffolding is in place (`kaggle/notebook.ipynb`, `scripts/run_on_kaggle.py`, `configs/kaggle.yaml`). Expected: BLEU-4 in the 0.10 – 0.14

range, CheXbert macro-F1 in the 0.20 – 0.35 range based on prior work.

2. **CheXbert clinical evaluation.** The runner `eval/chexbert_eval.py` is implemented and waiting for GPU compute.
3. **Hybrid retrieval.** Augment ColPali's visual MaxSim with sparse textual retrieval over the CheXpert label set, to fix the "rare specifics" failure mode of §7.2.

References

::: {#refs} :::

[^bib]: Full BibTeX in `report/references.bib`.

- Faysse et al. 2024. *ColPali: Efficient Document Retrieval with Vision Language Models*. arXiv:2407.01449.
- Google Health. 2025. *MedGemma Technical Report*. <https://huggingface.co/google/medgemma-1.5-4b-it>.
- Ranjit et al. 2023. *Retrieval Augmented Chest X-ray Report Generation using OpenAI GPT models*. arXiv:2305.03660.
- Bae et al. 2023. *EHRXQA / MIMIC-CXR-VQA*. NeurIPS Datasets and Benchmarks.
- Johnson et al. 2019. *MIMIC-CXR-JPG*. arXiv:1901.07042.
- Irvin et al. 2019. *CheXpert: A large chest radiograph dataset*. AAAI.
- Smit et al. 2020. *CheXbert*. EMNLP.
- Papineni et al. 2002. *BLEU*. ACL.
- Lin 2004. *ROUGE*. Text Summarization Branches Out.
- Zhang et al. 2020. *BERTScore*. ICLR.
- Radford et al. 2021. *CLIP*. ICML.
- Beyer et al. 2024. *PaliGemma*. arXiv:2407.07726.
- Miura et al. 2021. *Improving Factual Completeness and Consistency of Image-to-Text Radiology Report Generation*. NAACL.

Appendix A — Repository layout

<code>data_prep/</code>	Kaggle MIMIC-CXR preprocessor, Open-i downloader, patient-level split, VQA constructors (LLM + rule-based), report-section parser.
<code>retrieval/</code>	ColPali index + MaxSim search; CLIP index + cosine search.
<code>generation/</code>	MedGemma wrapper, Moondream2 wrapper, SmolVLM wrapper, nearest-neighbour stub.
<code>rag/</code>	Factory (chooses backends from config) and Mode A/B drivers.
<code>eval/</code>	BLEU/ROUGE/METEOR, BERTScore, CheXbert, VQA EM + token-F1 + per-type accuracy, qualitative-dump generator.

prompts/	Diff-friendly .txt templates for VQA generation, RAG / baseline report generation, RAG / baseline VQA.
configs/	kaggle.yaml (brief), local.yaml (Moondream2), local_stub.yaml (nearest-neighbour), local_smolvlm.yaml.
kaggle/	Notebook + setup README for headless GPU runs.
scripts/	run.sh, run_local.sh, run_on_kaggle.py, make_synthetic_outputs.py.
tests/	25 unit tests (parsers, metrics, generators).
outputs/	Run artefacts (metrics_*.json, mode_*.jsonl, qualitative_*.md).
report/	This document and references.bib.
app.py	Gradio web demo (http://localhost:7860).

Appendix B — Reproducibility

```
# Substitute stack (no creds needed):
pip install -r requirements.txt
bash scripts/run_local.sh           # Moondream2   (heavy)
CFG=configs/local_stub.yaml bash scripts/run_local.sh  # Stub (fast)

# Brief stack (needs Kaggle + HF + Anthropic):
pip install kaggle hf_xet
# put kaggle.json in ~/.kaggle/
python scripts/run_on_kaggle.py     # one-shot
```

All seeds fixed at 42; all configs version-controlled; all prompts in diff-friendly .txt files under prompts/; all results in outputs/ are reproduced from the same JSONL predictions.

Appendix C — Web demo

Launch `python app.py`; open <http://localhost:7860>. Two tabs:

- **Mode A — Report generation.** Pick an X-ray from the test dropdown or upload your own. The app runs both RAG and baseline systems and shows: input image, ground-truth report (when picking from the test set), RAG prediction, baseline prediction, and the top-k retrieved reports with similarity scores.
- **Mode B — VQA.** Same layout with a question text box. Selecting a pre-canned question populates the text box automatically.

The backend is config-driven (`CFG=configs/local_stub.yaml` by default); flipping `CFG=configs/kaggle.yaml` and pointing at an existing index folder gives the brief stack on the same UI.